

Restaurant Management System

Project Title: Restaurant Management System.

Course: CSC 478 – Software Engineering Capstone

Team Name: Error 404

Team Members: Haripriya Koneru, Krishna Kumar Nimmagadda, Venkata Sai Gowtham Panyam, Vansh Parihar, Brijeshkumar Mansukhbhai Raiyani, Pratham Patel, Vijaya Venkateswara Rao Rowthu

Document: Test Plan and Test Results

Testing Documentation

This Test Plan will confirm that the Restaurant Management System (RMS) satisfies all the functional and system requirements (UR-xx, SR-xx) and will work properly in normal use conditions. Manual testing is performed using black-box test techniques on Google Chrome.

Each test case includes:

- Test inputs
- Expected outputs
- Rationale
- Actual result (Pass / Fail / Pending)

1.1 Testing Approach

Testing is manual black-box testing focused on validating user-facing features across all role-specific dashboards (Host, Waiter, Chef, Manager, Owner) and on verifying system-level functionality (sessions, permissions, order flow, reports). No internal code inspection is used in this phase.

Testing goals:

- Confirm each requirement behaves correctly.
- Validate UI workflows and role-based access.
- Test menu management, order handling, and order lifecycle.
- Verify input validation and user feedback.

1.2 Test Environment

Category	Details
Operating System	Windows 10 / macOS (latest)
Browsers	Google Chrome (latest), Microsoft Edge (latest)
Framework	React + Vite (frontend), Node.js + Express (backend), MongoDB
Testing Type	Manual, Black-box
Repositories	Frontend: https://github.com/Patel-Pratham-3492/Error_404 Backend: https://github.com/Patel-Pratham-3492/Error_404_server

Host Role Tests (UR-Hx)

TC-H-01 — UR-H1 — Host Login

Inputs: Open front-of-house client → Navigate to Login → Enter host credentials → Click Login

Expected output: Session cookie set, host redirected to Host Dashboard; host UI shows seating controls.

Rationale: Only authorized hosts must access table management.

Actual result: Pass

TC-H-02 — UR-H2 — View Table Status

Inputs: From Host Dashboard, view table layout/status panel

Expected output: All tables display as Open or Occupied; status updates are current.

Rationale: Host must see table availability to seat guests.

Actual result: Pass

TC-H-03 — UR-H3 — Assign Table to Guests

Inputs: Select an open table → Click “Assign” → Enter party size and name → Confirm

Expected output: Table marked Occupied; waiter assignment (if auto-assign) updated; UI shows guest info.

Rationale: Ensures quick, accurate seating and record keeping.

Actual result: Pass

TC-H-04 — UR-H4 — Mark Table Occupied

Inputs: On seating or manual action, Host clicks “Mark Occupied” for table X

Expected output: Table state becomes Occupied and visible to Waiter/Manager/Owner dashboards in real time.

Rationale: Prevents double seating.

Actual result: Pass

TC-H-05 — UR-H5 — Mark Table Open After Exit

Inputs: When guests leave, Host clicks “Mark Open” on table Y

Expected output: Table state becomes Open; visible across UI; table available for new assignments.

Rationale: Frees tables and updates availability.

Actual result: Pass

Waiter Role Tests (UR-Wx)

TC-W-01 — UR-W1 — Waiter Login

Inputs: Login as waiter via POST /api/login → open Waiter Dashboard

Expected output: Session established, waiter sees assigned tables.

Rationale: Authenticates waiters and ensures accountability.

Actual result: Pass

TC-W-02 — UR-W2 — Input Customer Order

Inputs: Select table → Create new order: choose items and quantities → Submit order

Expected output: Order created in DB, visible on Waiter and Chef dashboards, order status = pending.

Rationale: Core functionality for ordering.

Actual result: Pass

TC-W-03 — UR-W3 — Modify Order for Preferences

Inputs: Select an existing order → Edit item modifiers (e.g., “no onion”) → Save changes

Expected output: Order updated in DB and reflected on Chef UI with modifiers.

Rationale: Accommodate customer preferences.

Actual result: Pass

TC-W-04 — UR-W4 — Add Additional Orders Later

Inputs: For a table with an existing order, create a new order or append items (dessert) → Submit

Expected output: New items appended to order history; chef receives new items as separate line items.

Rationale: Workflow flexibility for multi-phase dining.

Actual result: Pass

TC-W-05 — UR-W5 — Verify Table Mapping on Order

Inputs: Create order for table 7 → Observe order metadata and delivery tag

Expected output: Order has tableNumber=7; Waiter and Chef see table association.

Rationale: Ensure correct delivery routing.

Actual result: Pass

TC-W-06 — UR-W6 — Track Drinks Separately

Inputs: Add drinks as separate order category or flag drinks on order form → Submit

Expected output: Drinks appear with a separate drinks list/indicator; system tracks drink refill count.

Rationale: Drink service tracking and refills.

Actual result: Fail

TC-W-07 — UR-W7 — Take Payment

Inputs: On completed order, open payment modal → Enter bill amount and tip → Confirm payment

Expected output: Payment recorded in DB; order marked Paid; receipt displayed.

Rationale: Complete checkout process.

Actual result: Pass

TC-W-08 — UR-W8 — Restrict Waiter View to Assigned Tables

Inputs: Login as waiter A assigned only tables 1–4; attempt to view table 8 → UI action or URL navigation

Expected output: Waiter only sees assigned tables; forbidden access to others (403 or UI restriction).

Rationale: Enforces accountability and reduces confusion.

Actual result: Pass

Chef Role Tests (UR-Cx)

TC-C-01 — UR-C1 — Chef Login

Inputs: Login as chef → access Chef Dashboard

Expected output: Session established, chef UI shows order queue.

Rationale: Secure access to kitchen features.

Actual result: Pass

TC-C-02 — UR-C2 — View Incoming Orders

Inputs: Have waiter submit an order → Observe Chef UI refresh/queue

Expected output: New order appears in chef queue immediately with full item details.

Rationale: Timely cook order intake.

Actual result: Pass

TC-C-03 — UR-C3 — Prioritize Orders

Inputs: Chef reorders queue or flags an order as high priority → Save

Expected output: Order state/position updated and visible to chef; optionally informs wait staff.

Rationale: Manage kitchen workload in busy periods.

Actual result: Fail, No Functionality

TC-C-04 — UR-C4 — Mark Item In-Progress

Inputs: On an order, chef clicks “Preparing” on an item → Save

Expected output: Item status = in-progress; waiters see in-progress status.

Rationale: Communication on cooking status.

Actual result: Pass

TC-C-05 — UR-C5 — Mark Item Complete

Inputs: After cooking, chef clicks “Done” → Save

Expected output: Item status = done; waiter receives notification to serve.

Rationale: Signal readiness for serving.

Actual result: Pass

Manager Role Tests (UR-Mx)**TC-M-01 — UR-M1 — Manager Performs Waiter Actions**

Inputs: Login as manager → place an order or take payment on behalf of waiter → submit

Expected output: Manager able to perform waiter flows; actions recorded as performed by manager.

Rationale: Manager backup capability during rush.

Actual result: Pending

TC-M-02 — UR-M2 — Add/Remove Workers

Inputs: Manager navigates to Hire Staff/Fire Staff → Add new staff with role 'waiter' → Save → Remove demo staff → Confirm → Email to the user Fire/Hire

Expected output: New user created in Users collection; removed user no longer able to login.

Rationale: Maintain accurate staff roster.

Actual result: Pass

TC-M-03 — UR-M3 — Assign Tables to Waiters

Inputs: Manager opens Table Assignment → Assign table 5 to Waiter B → Save

Expected output: Table 5 now visible to Waiter B; previous assignment updated.

Rationale: Clear allocations of service zones.

Actual result: Pass

TC-M-04 — UR-M4 — Mark Prices for Specials

Inputs: Manager edits price on menu item → toggles special flag → Save

Expected output: Menu shows special price; displays special badge; price effective immediately.

Rationale: Run daily promotions.

Actual result: Pass

TC-M-05a — UR-M5a — Add Menu Item

Inputs: Manager enters new menu item details → Save (Name, Price, Category)

Expected output: New item visible in menu (manager and waiter UIs).

Rationale: Add seasonal or new dishes.

Actual result: Pass

TC-M-05b — UR-M5b — Remove Menu Item

Inputs: Manager deletes item ID → Confirm deletion

Expected output: Item removed from menu; menus update across dashboards.

Rationale: Remove discontinued dishes.

Actual result: Pass

TC-M-05c — UR-M5c — Change Menu Prices

Inputs: Manager edits price on item → Save → Verify price reflected on menu and in new orders

Expected output: New price used for subsequent orders and display.

Rationale: Pricing control.

Actual result: Pass

Owner Role Tests (UR-Ox)

TC-O-01 — UR-O1 — Owner All-Access

Inputs: Login as owner → Navigate through manager features (staff), reports, and high-level settings

Expected output: Owner has access to manager features plus owner-only pages like reports.

Rationale: Owner oversight and control.

Actual result: Pending

TC-O-02 — UR-O2 — Add/Remove Managers

Inputs: Owner adds manager account → assign privileges → remove manager → confirm rules enforcement

Expected output: Manager accounts created/removed correctly; RBAC enforced.

Rationale: Control leadership roles.

Actual result: Pass

TC-O-03 — UR-O3 & UR-O4a/b — Access Reports (Item quantities & Revenue)

Inputs: Owner navigates to Reports → Select date range → Generate 'Item Quantity' and 'Total Revenue' reports

Expected output: Reports show correct aggregated counts and sums for selected date range; export option available.

Rationale: Business performance analysis.

Actual result: Pass

System Role / Cross-Cutting Tests (SR-x)

TC-SR-01 — SR-1 — System Login Works for All Roles

Inputs: Try login with representative credentials for Host, Waiter, Chef, Manager, Owner

Expected output: Each user is authenticated and routed to role-specific dashboard; session cookie present.

Rationale: Fundamental security and role routing.

Actual result: Pass

TC-SR-02 — SR-2 — Role-Based Permissions Enforced

Inputs: Attempt Manager-only action (e.g., delete menu) from waiter account via UI and direct API call

Expected output: UI should hide action; API returns 403 Forbidden if attempted.

Rationale: Enforce least privilege.

Actual result: Pass

TC-SR-04 — SR-4 — Role Switching Support

Inputs: User with multiple roles (e.g., Manager+Waiter) selects alternative role in UI → UI changes features accordingly

Expected output: Session updates role context; new UI shows appropriate permissions.

Rationale: Flexibility for multi-role staff.

Actual result: Fail, No Functionality

TC-SR-05 — SR-5 — Orders Can Be Submitted Anytime

Inputs: Submit orders during peak simulation (many rapid requests)

Expected output: System accepts orders; queue increases and processes in order; no UI deadlock.

Rationale: Reliability during busy service.

Actual result: Pass

TC-SR-06 — SR-6 — Orders Support Categories

Inputs: Create orders containing starters, mains, desserts, drinks → Save

Expected output: Order items categorized and displayed properly in Chef UI (possibly grouped).

Rationale: Kitchen organization.

Actual result: Pass

TC-SR-07 — SR-7 — Real-Time Orders to Chef UI

Inputs: Waiter submits order → Observe Chef UI for immediate arrival (websocket/refresh)

Expected output: Orders appear in near real-time without manual refresh (or with short refresh).

Rationale: Minimize processing delay.

Actual result: Pass

Note: The screen refresh after 30sec is causing delay in showing real time orders

TC-SR-08 — SR-8 — Order Status Tracking

Inputs: Waiter creates order → Chef marks items in-progress then done → Waiter views status updates

Expected output: Status transitions visible and timestamped.

Rationale: Track order lifecycle reliably.

Actual result: Pass

TC-SR-09 — SR-9 — Table Status Tracking

Inputs: Host marks tables occupied/open → Waiter and manager dashboards reflect change

Expected output: Table status synchronized across dashboards.

Rationale: Prevent double bookings.

Actual result: Pass

TC-SR-10 — SR-10 — Waiter-Table Assignment Tracking

Inputs: Manager assigns table to waiter → Waiter logs in and confirms assigned tables list

Expected output: Assignment persisted and visible; actions attributable to waiter.

Rationale: Accountability.

Actual result: Pass

TC-SR-12 — SR-12 — Tipping Supported

Inputs: During payment, enter tip amount → Save

Expected output: Tip stored and included in total revenue report.

Rationale: Standard hospitality practice.

Actual result: No functionality

TC-SR-13 — SR-13 — Report Generation

Inputs: Owner requests reports for time range → export CSV/print

Expected output: Reports compile correctly and export as expected.

Rationale: Business insights and compliance.

Actual result: Fail

Summary of Failed / Problematic Tests

Failed / Problematic test cases found during manual black-box testing:

- **TC-W-06 — Track Drinks Separately — Fail**
Drinks are not tracked separately as a distinct category or list; refill counts not tracked.
- **TC-C-03 — Prioritize Orders — Fail (No functionality)**
Chef cannot reorder or flag orders as priority.
- **TC-SR-04 — Role Switching Support — Fail (No functionality)**
Users with multiple roles cannot switch role contexts in the UI/session.
- **TC-SR-12 — Tipping Supported — No functionality**
Payment flow does not support tip entry or storing tip amount.
- **TC-SR-13 — Report Generation — Fail**
Owner reports (item quantities, total revenue) fail to generate/export.
- **Partial / Performance note — TC-SR-07 Real-Time Orders**
Marked as **Pass**, but chef receives new orders only after a 30s screen refresh interval — causes unacceptable delay for “real-time” requirement.
- **Numeric precision issue (Revenue values)**
Total revenue shows 9 decimal places in manager/owner views instead of being rounded/truncated.

Bugs to fix / Missing features

- **Drinks not tracked separately**

Drinks are mixed with food items and refill counts are not tracked.

Severity: Medium

- **Chef cannot prioritize orders**

No UI/API to flag or reorder orders by priority.

Severity: High

- **Role switching not supported**

Users with multiple roles cannot switch active role/session in UI.

Severity: Medium

- **Tipping not supported in payment flow**

Payment modal has no tip field and tips are not stored or reported.

Severity: Medium

- **Report generation/export fails for Owner**

Item quantity and total revenue reports fail to generate or export.

Severity: Critical

- **Real-time orders delayed (~30s refresh)**

Chef sees new orders only after ~30s polling interval; needs near-real-time.

Severity: Medium

- **Revenue shows excessive decimal places**

Total revenue displays up to 9 decimal places instead of being rounded to 2.

Severity: Medium